

AI Extension – Playwright Python

Step 1:

Prompt 1: (In Ask Mode)

src > scripts > JS prompts.js > ...

```
4 export const DEFAULT_PROMPTS = {
415 CUCUMBER_WITH_PLAYWRIGHT_PYTHON_STEPS: `
474
475 @given('I open the login page')
476 def open_login_page(context):
477     """Navigate to the login page"""
478     page.goto(`${pageUrl}`)
479
480 @when('I type "{username}" into the Username field')
481 def enter_username(context, username: str):
482     """Enter username into the username field
483
484     Args:
485         context: Behave context
486         username: Username to enter
487     """
488     username_input = page.locator('#username')
489     username_input.wait_for(state='visible')
490     username_input.fill(username)
491
492 @when('I type "{password}" into the Password field')
493 def enter_password(context, password: str):
494     """Enter password into the password field
495
496     Args:
497         context: Behave context
498         password: Password to enter
499     """
500     password_input = page.locator('#password')
501     password_input.wait_for(state='visible')
502     password_input.fill(password)
503
504 @when('I click the Login button')
505 def click_login(context):
506     """Click the login button"""
507     login_button = page.locator('button:has-text("Login")')
508     login_button.click()
```

CHAT

ANALYZE JAVA SELENIUM IMPLEMENTATION

Read my entire workspace to understand and analyze how typescript Playwright is implemented in codebase and tell what are all the impacted files.

I want to add playwright python same like implemented for playwright typescript

Prefacing tool calls and updates

Searching the workspace for all references to "Playwright" to map where TypeScript Playwright is used and what to extend for Python support.

Searching for Playwright and analyzed its implementation

Plan: summarize how Playwright TypeScript is wired, list all impacted files, confirm Playwright Python status, and give exact steps to add/extend Python support.

Summary

- Playwright TypeScript is implemented as prompt templates + UI routing + syntax highlighting. The pipeline: UI selection → chat.js chooses prompt keys → prompts.js provides prompt text → LLM APIs generate code.
- Good news: Playwright Python support already exists in the repo (prompts + routing present). You can use it as-is or extend it with additional templates.

Impacted files

ask a question about your code or project

Local Default Approvals

Ln 1, Col 1 Spaces: 2 UTF-8 LF JavaScript

Step 2:

Prompt 2: (In Ask mode)

src > scripts > JS prompts.js > ...

```
4 export const DEFAULT_PROMPTS = {
415 CUCUMBER_WITH_PLAYWRIGHT_PYTHON_STEPS: `
474
475 @given('I open the login page')
476 def open_login_page(context):
477     """Navigate to the login page"""
478     page.goto(`${pageUrl}`)
479
480 @when('I type "{username}" into the Username field')
481 def enter_username(context, username: str):
482     """Enter username into the username field
483
484     Args:
485         context: Behave context
486         username: Username to enter
487     """
488     username_input = page.locator('#username')
489     username_input.wait_for(state='visible')
490     username_input.fill(username)
491
492 @when('I type "{password}" into the Password field')
493 def enter_password(context, password: str):
494     """Enter password into the password field
495
496     Args:
497         context: Behave context
498         password: Password to enter
499     """
500     password_input = page.locator('#password')
501     password_input.wait_for(state='visible')
502     password_input.fill(password)
503
504 @when('I click the Login button')
505 def click_login(context):
506     """Click the login button"""
507     login_button = page.locator('button:has-text("Login")')
508     login_button.click()
```

CHAT

ANALYZE JAVA SELENIUM IMPLEMENTATION

Implement the Playwright Python in the impacted files

Modified language support and updated language checks

Planned steps

- Update chat.js so Python language value (py) from the UI is recognized and Playwright/Python defaults to page-object generation.
- Provide exact patches (with surrounding context) you can apply directly.

Edits to make

1. Replace the "no generation mode selected" fallback so it defaults to a Page Object for Playwright TypeScript/Python and Selenium Java.

Replace this block (3 lines before/after shown):

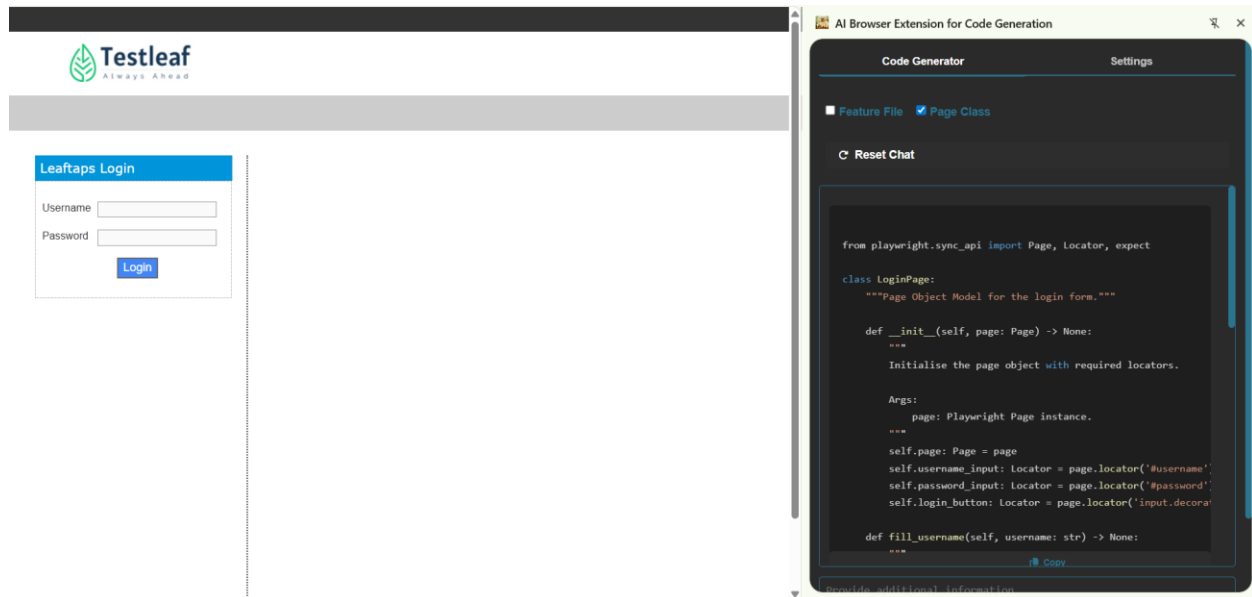
```
// Validate that at least one option is selected
if (!isFeatureChecked && !isPageChecked) {
    console.warn('No generation mode selected. Defaulting to Page Object generation')
    // Default fallback to page object generation
    if (this.isJavaSelenium(lang, eng)) {
        promptKeys.push('SELENIUM_JAVA_PAGE_ONLY');
    }
    return promptKeys;
}
```

Implement the Playwright Python in the impacted files

Local Default Approvals

Prompt 3: (In Agent mode)





Step 6:

Playwright +Python code created below for Page.

```
from playwright.sync_api import Page, Locator, expect

class LoginPage:
    """Page Object Model for the login form."""

    def __init__(self, page: Page) -> None:
        """
        Initialise the page object with required locators.

        Args:
            page: Playwright Page instance.
        """
        self.page: Page = page
        self.username_input: Locator = page.locator('#username')
        self.password_input: Locator = page.locator('#password')
        self.login_button: Locator = page.locator('input.decorativeSubmit[value="Login"]')

    def fill_username(self, username: str) -> None:
        """
        Fill the username field.

        Args:
            username: The username to enter.
        """
        expect(self.username_input).to_be_visible()
        self.username_input.fill(username)

    def fill_password(self, password: str) -> None:
```

```
"""
Fill the password field.

Args:
    password: The password to enter.
"""
expect(self.password_input).to_be_visible()
self.password_input.fill(password)

def click_login(self) -> None:
    """
    Click the login button to submit the form.
    """
    expect(self.login_button).to_be_enabled()
    self.login_button.click()

def login(self, username: str, password: str) -> None:
    """
    Perform a full login action by filling credentials and submitting.

    Args:
        username: The username to enter.
        password: The password to enter.
    """
    self.fill_username(username)
    self.fill_password(password)
    self.click_login()
```